

REV 1.1 / SITL-VERIFIED / 2026

AUTONOMOUS UAV / COMPANION-COMPUTER SAFETY

Drone Safety System

Verified dispatch & failsafe arbitration for fully autonomous flight.

0.4 m

TERMINAL ACCURACY

8 / 8

ACCEPTANCE CHECKS

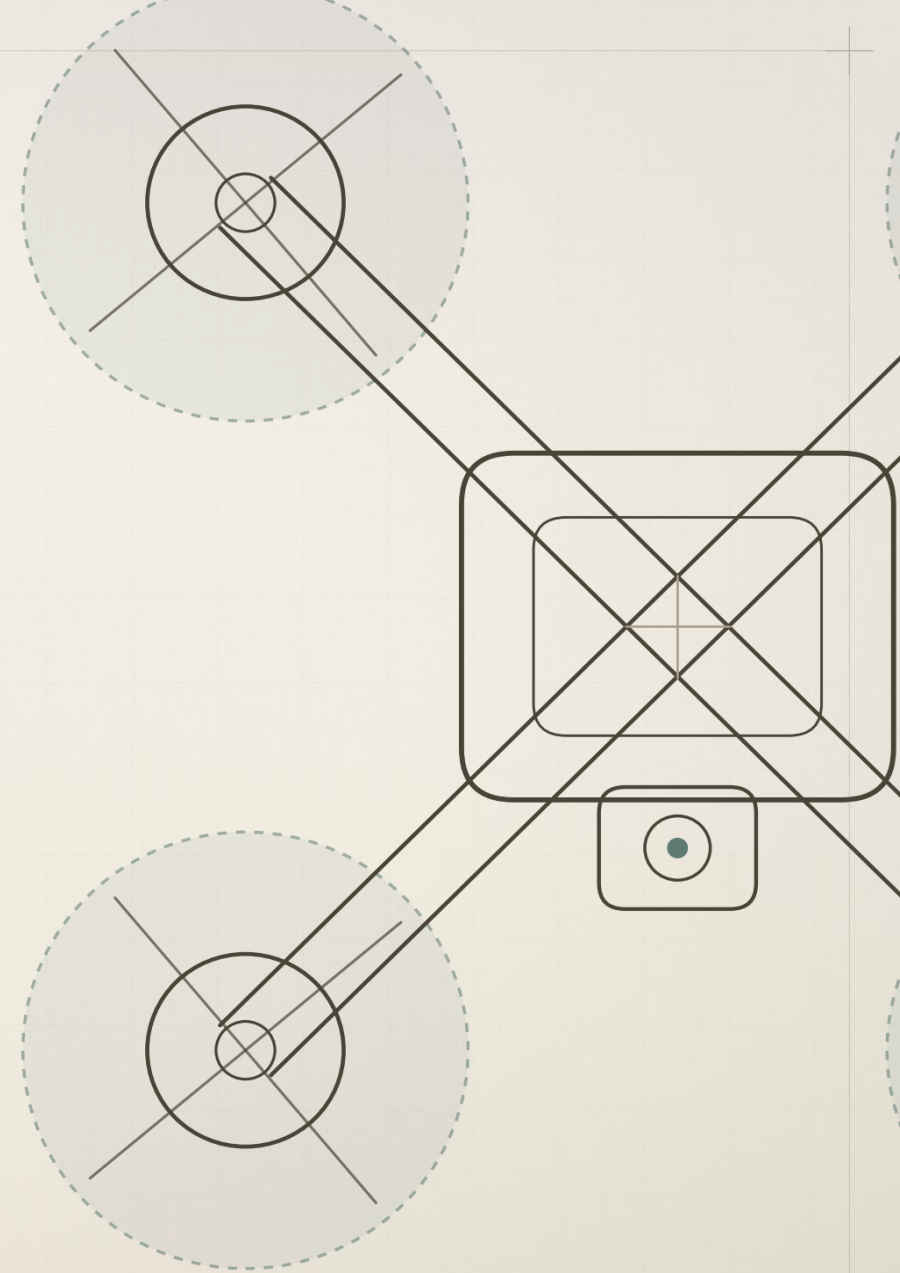
54

UNIT TESTS

2

PATENTS

github.com/SV-1411/drone



What this system is

A companion-computer stack that turns one network trigger into a complete autonomous mission — fly to a GPS target, hover, and return — with no pilot, and aviation-grade safety woven through every phase.

Trigger-to-flight

POST a coordinate; the drone arms, takes off, navigates, hovers and returns on its own.

Verified commands

Every flight-mode change is confirmed from the autopilot's own telemetry, never assumed.

Arbitrated failsafes

Battery, GPS, geofence & timeout hazards resolve to one debounced action.

Provable safety

A landing interlock makes it impossible to start a mission on an airborne drone.

Autonomy removes the operator who catches failures

- **The autopilot can lie.** A mode-change is reported accepted while the autopilot silently stays in its old mode — seen on real ArduCopter.
- **No human is watching.** In autonomous dispatch nobody notices a dropped command — and the same path carries emergency RTL/LAND.
- **Missions can overlap.** Treating mission N's end as license to start N+1 can arm a still-airborne aircraft — undefined & hazardous.
- **Sensors glitch.** One bad GPS sample, taken naively, triggers an instant land-in-place — over a road, water, or a crowd.

The core insight

“A command issued is not a command executed.”

So we treat the autopilot as untrusted: demand telemetry-confirmed proof of every transition — then guarantee the queue can never hand off an airborne drone.

Four cleanly separated parts

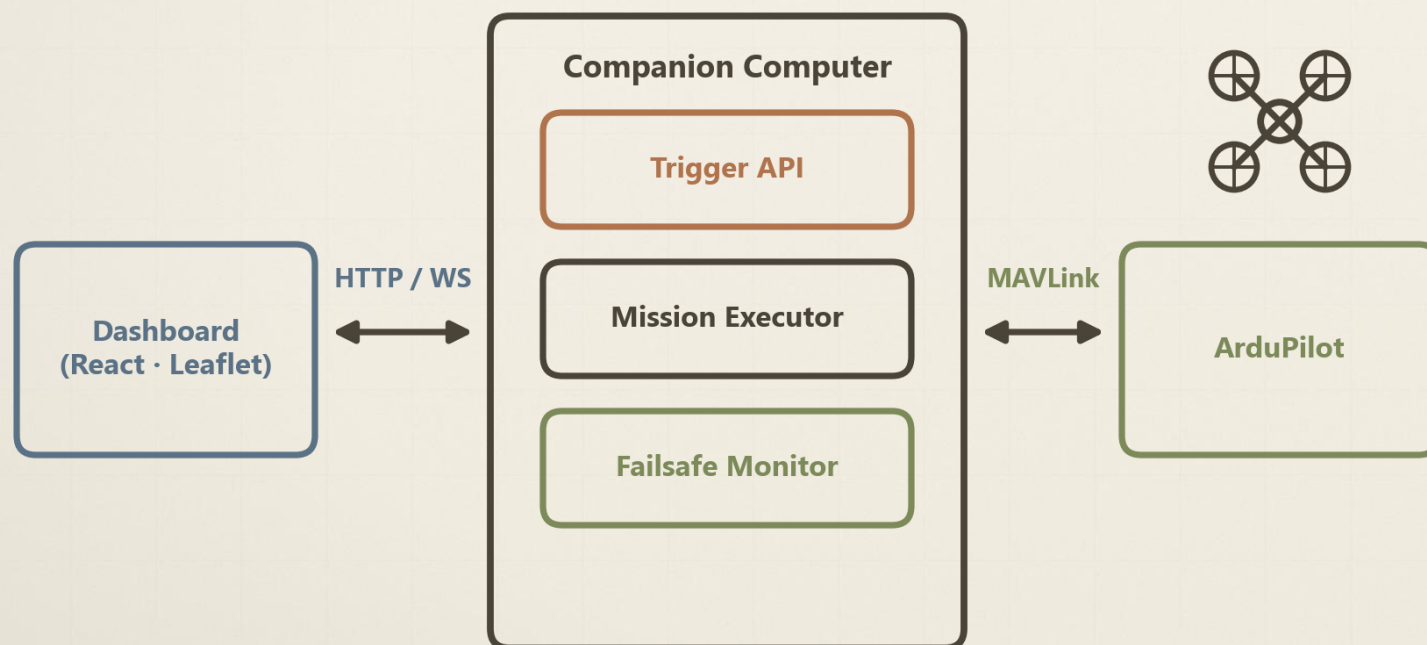


Figure 1 · System architecture

- **Trigger API** — validate, queue, stream.
- **Executor** — the 12-state flight machine.
- **Failsafe monitor** — 1 Hz hazard arbiter.
- **Dashboard** — live Leaflet map.

The full system — hardware & software

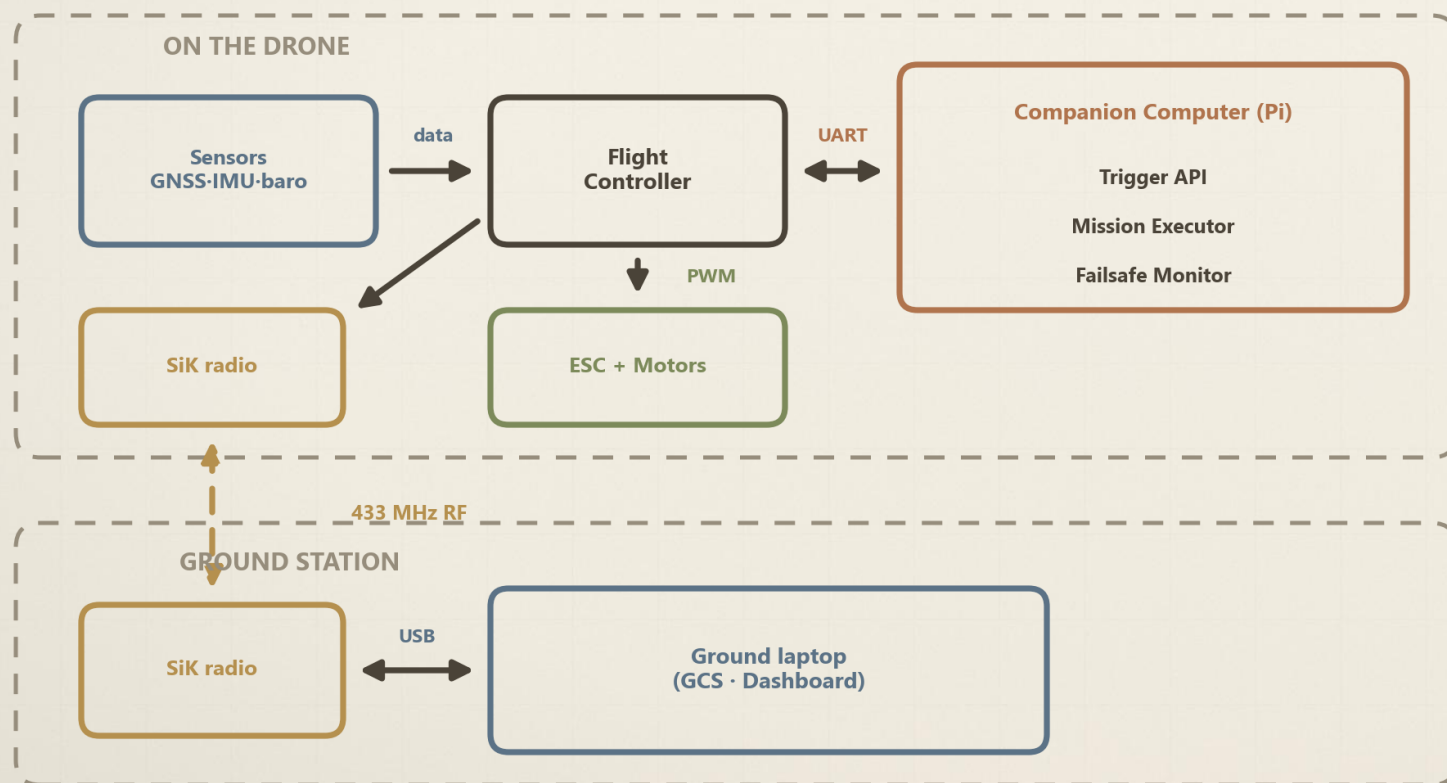
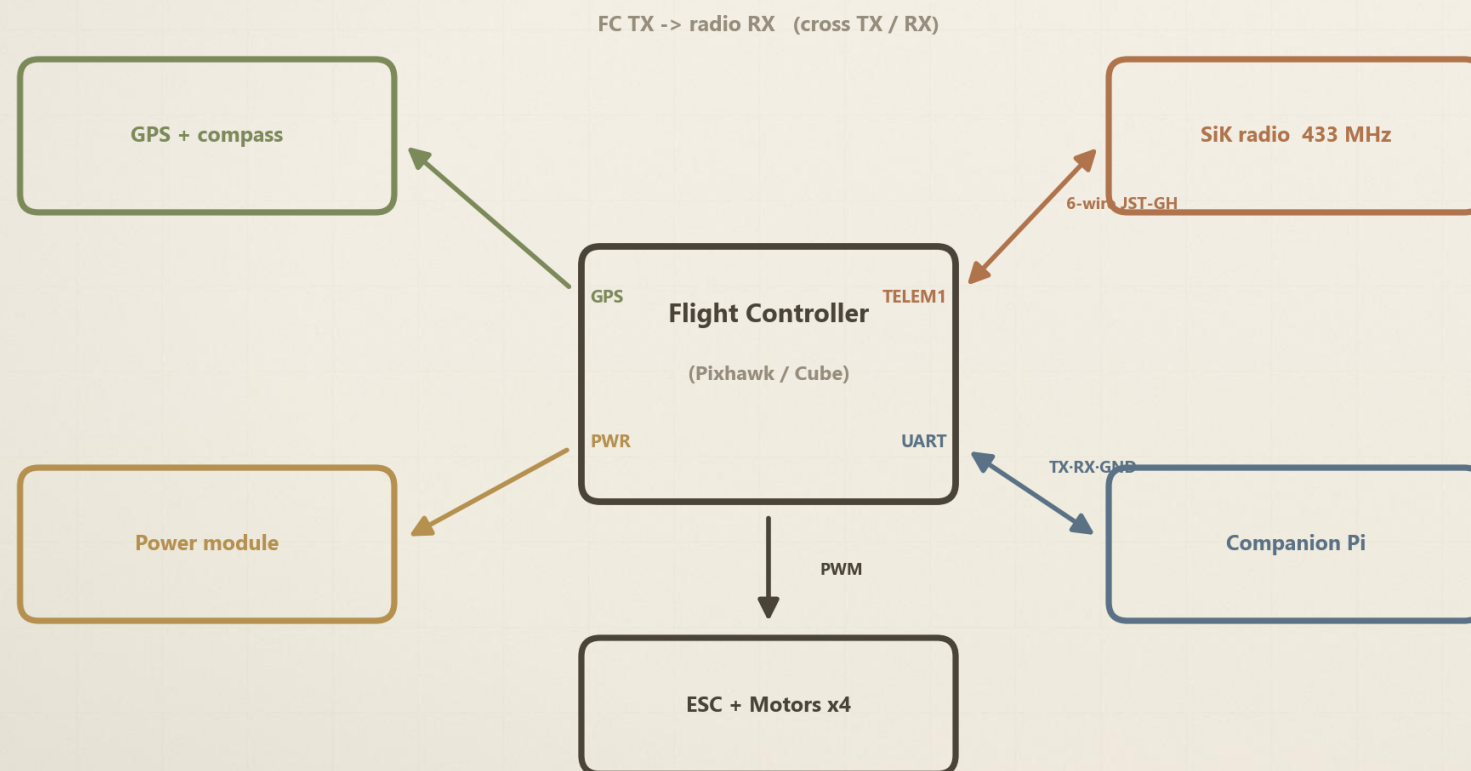


Figure 2 · End-to-end system (drone + ground)

- **On the drone** — sensors → flight controller → companion Pi → motors.
- **Companion** — runs the Trigger API, executor & failsafe.
- **Link** — SiK 433 MHz radio to the ground laptop.
- **Ground** — GCS / dashboard for monitoring & override.

How the radio & companion are wired in



- **TELEM1** → **SiK radio** — 6-wire JST-GH; cross FC TX → radio RX.
- **UART** → **Companion** — TX·RX·GND; runs the autonomy stack.
- **GPS & power** — to their dedicated ports.
- **MAIN OUT** → **ESCs** — PWM to the four motors.

Figure · Flight-controller wiring (SiK 433 + companion)

How the system was built & proven

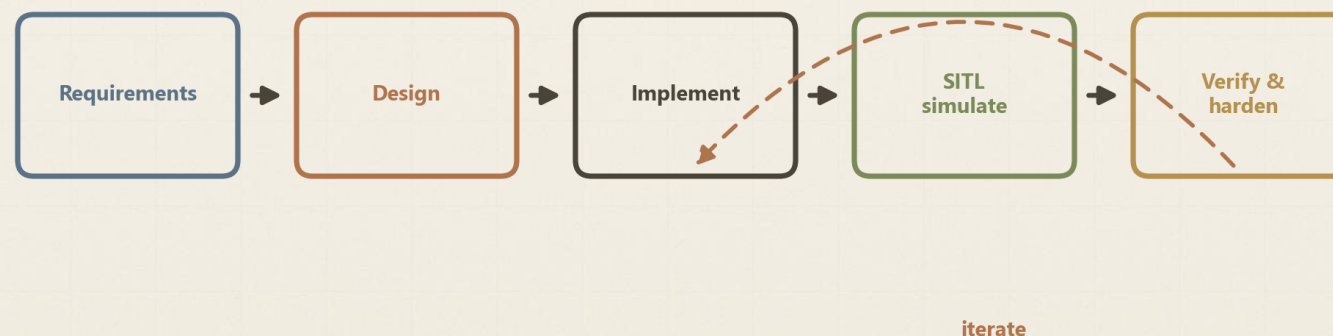
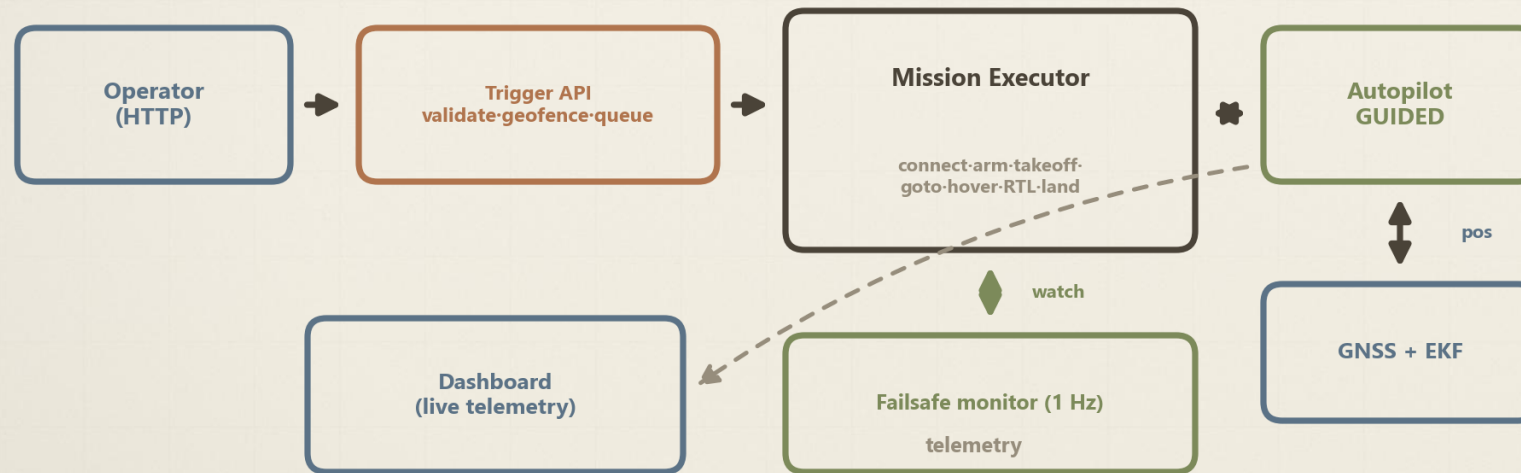


Figure 3 · Iterative, test-driven development method

- **Test-driven** — design → implement → simulate in SITL → verify with 37 unit + E2E tests → harden, repeat.
- **Evidence-led** — every safety claim is pinned by an automated test before it enters the patent drafts.

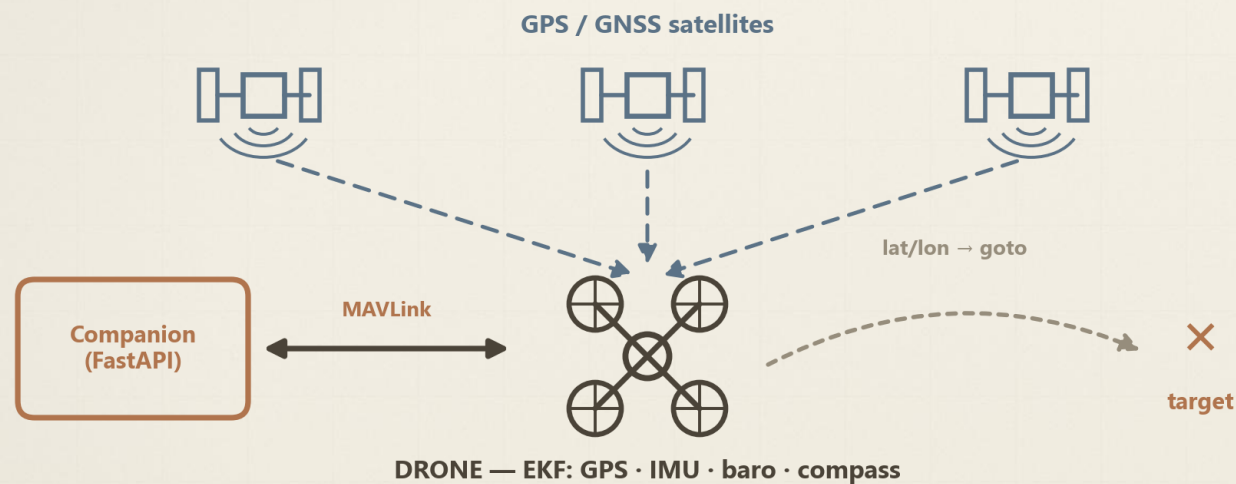
End-to-end: how it all flows



- **Input** — operator POSTs a coordinate.
- **Process** — API validates & queues; executor flies the phases.
- **Control** — autopilot in GUIDED on GNSS-fused position.
- **Feedback** — live telemetry out; failsafe watches throughout.

Figure 4 • Working & data flow, input to flight

How it knows where it is: GPS, not cell towers



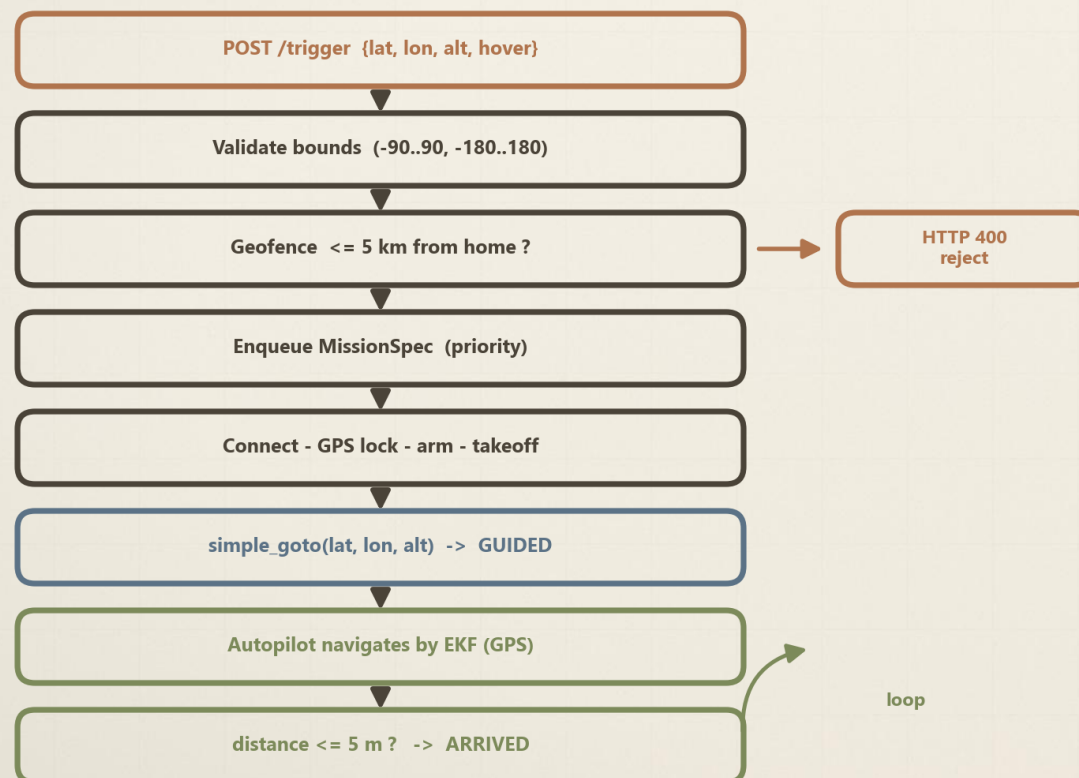
- **Positioning** — GNSS + IMU + baro + compass, fused onboard.
- **Data link** — MAVLink over radio / WiFi / LTE.

Figure 2 · Positioning by satellite GNSS + sensor fusion

Cell / 5G is only an optional link — never how the drone locates itself.

Positioning is satellite GNSS.

How the drone receives coordinates



- **Coordinate in** — one POST with lat/lon (+ optional alt, hover).
- **Checked twice** — value bounds, then geofence vs home (else 400).
- **Queued** — as a MissionSpec; missions run strictly serially.
- **Handed over** — as a GUIDED-mode goto to the autopilot.
- **Closed-loop** — flies until within 5 m (haversine distance).

Figure 6 • From HTTP request to a navigated target

How it reaches the destination

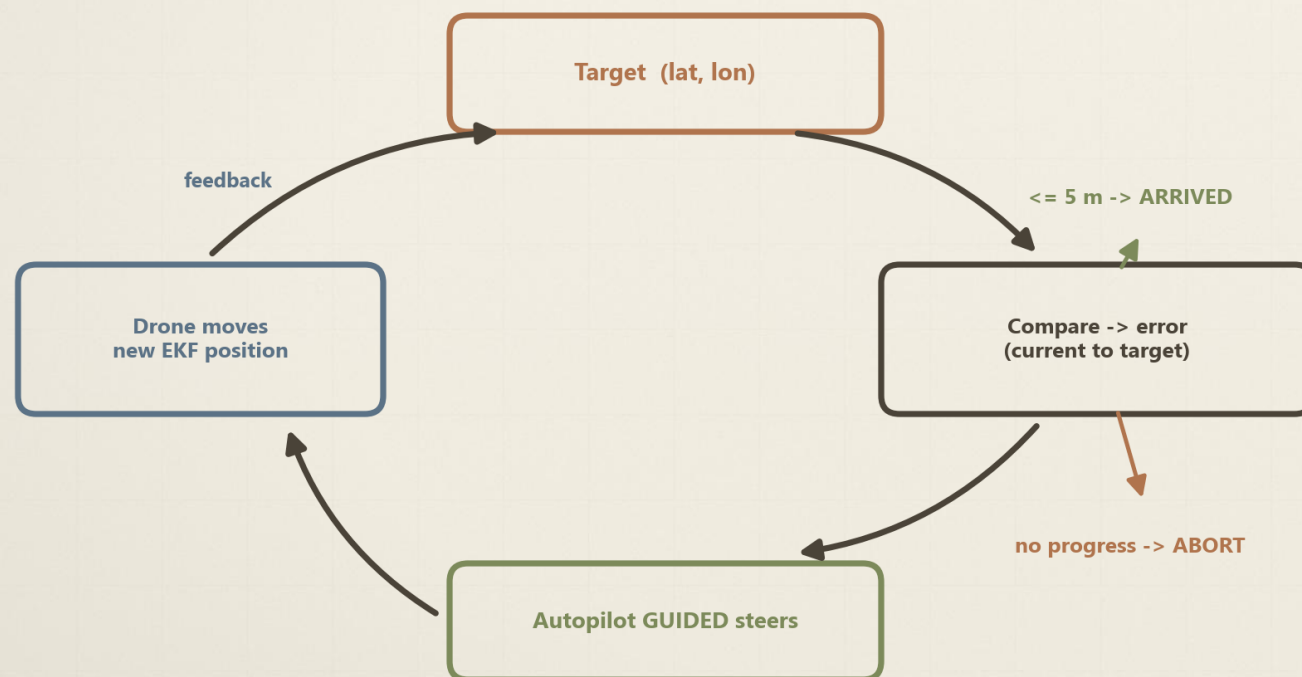
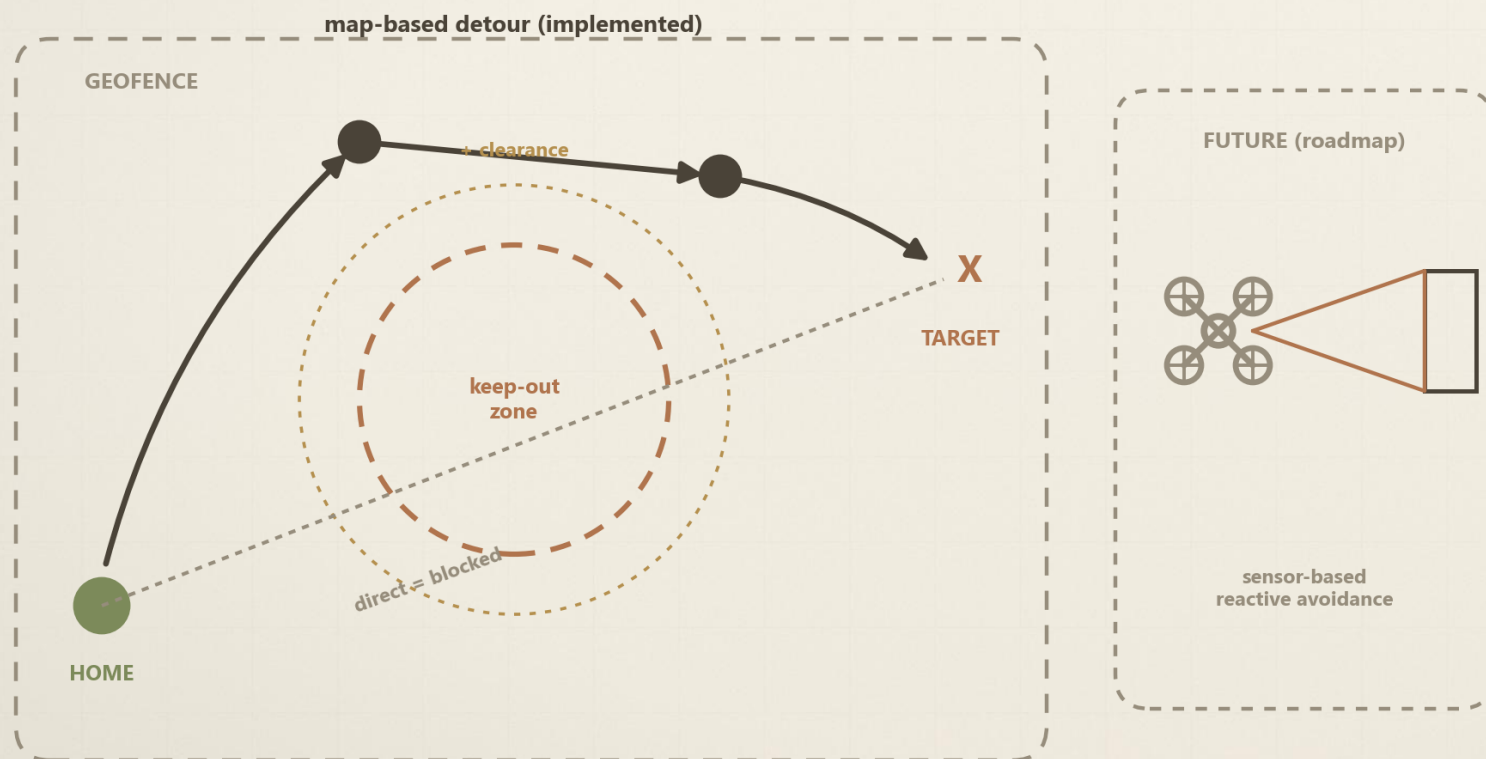


Figure 7 • The goto control loop

- **Closed loop** — compares EKF position to target every second.
- **Autopilot steers** — GUIDED mode corrects continuously.
- **Arrival** — declared within 5 m tolerance (RTK → ~0.3 m).
- **Stall guard** — no progress in N s → abort the leg safely.

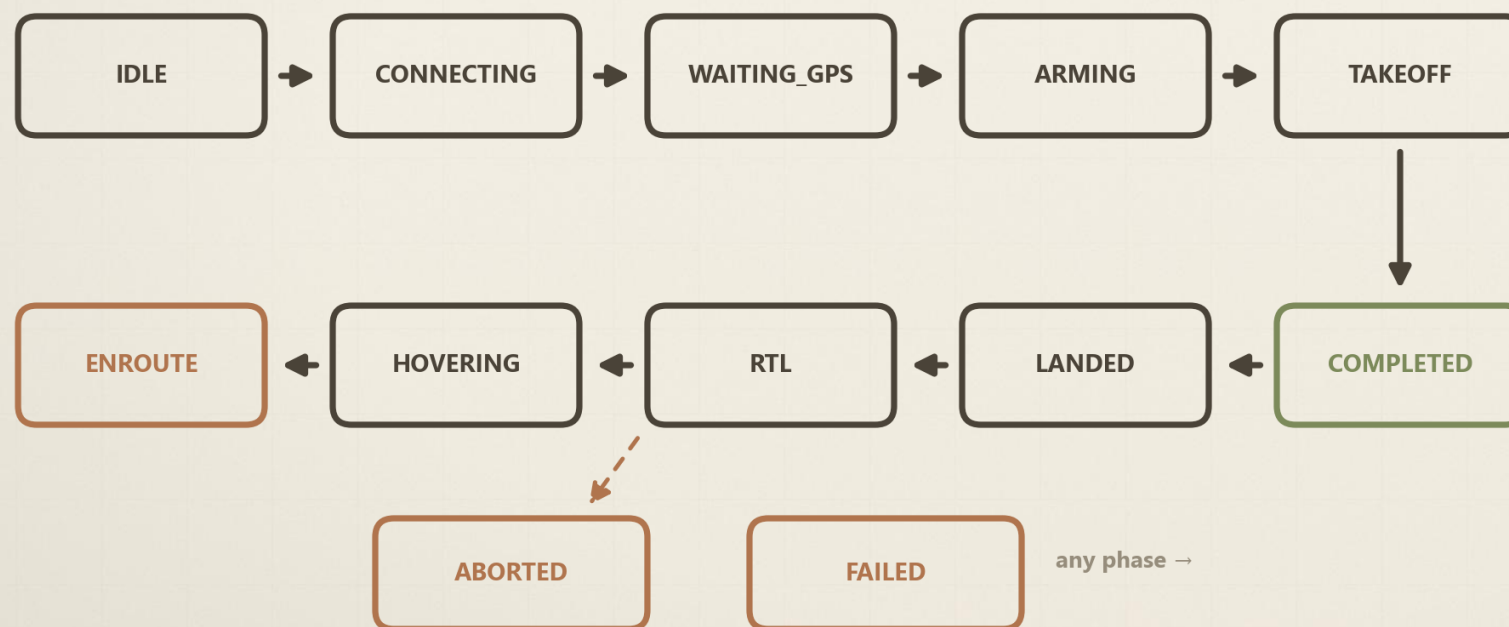
Obstacle avoidance — map-based, with a sensor roadmap



- **Map-based avoidance** — routes around configured keep-out zones (implemented, unit-tested).
- **Detour planning** — a blocked leg gets waypoints at a safe lateral offset.
- **Altitude + geofence** — flies above ground obstacles, inside a bounded area.
- **Failsafe abort** — battery / GPS / link → RTL or LAND.
- **Roadmap** — sensor-based reactive avoidance on Copter 4.x.

Figure 8 • Avoidance — implemented (map-based) + roadmap (sensors)

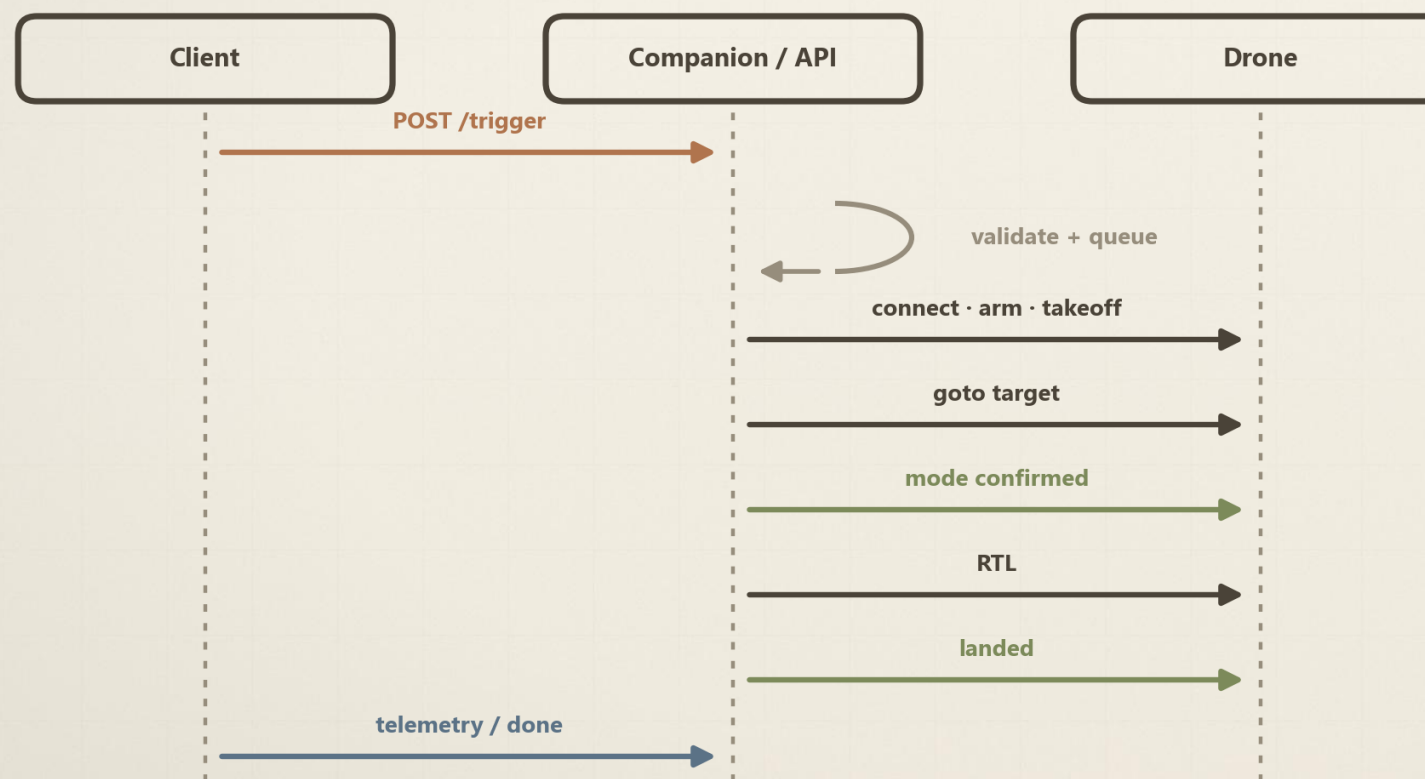
A 12-state flight machine — every step logged



- **Happy path** — arm → take off → enroute → hover → RTL → land → done.
- **Any phase** can divert safely to ABORTED or FAILED.
- **Abort blocks** until disarm — never hand off an airborne drone.

Figure 3 • Mission state machine

From network trigger to touchdown



- **1 · Trigger** — validated at the edge vs the geofence.
- **2 · Queue** — bounded, strictly serial.
- **3 · Execute** — connect, GPS lock, arm, climb.
- **4 · Verify** — each mode change confirmed.
- **5 · Return** — hover, RTL, land, log incident.

Figure 4 · Mission message sequence

Verified flight-mode delivery + landing-interlocked dispatch

Treat the autopilot as untrusted; prove every command; never overlap missions.

- **Confirmed setter** — issue via high-level API, then re-issue as COMMAND_LONG / DO_SET_MODE and SET_MODE until telemetry confirms.
- **Cross-action fallback** — if RTL won't confirm, command LAND through the same verified routine, and vice versa.
- **Abort guarantee** — every abnormal exit blocks until the vehicle reports disarmed (or a 240 s bound).
- **Pre-flight interlock** — refuse to arm/take off while armed — violation becomes impossible, not just improbable.
- **Edge geofence** — reject out-of-fence targets & waypoints before flight.

8 claims · IPC B64U 10/13, G05D 1/00, G08G 5/00 · IPO Form 2 complete-specification draft

The verified mode setter, in practice

```
# issue, then confirm from the autopilot's OWN telemetry
v.mode = target          # 1) high-level setter
while now < deadline:
    if v.mode.name == target:  # confirmed!
        return True
    raw_set_mode(target)      # 2) COMMAND_LONG
    set_mode_send(target)     # 3) SET_MODE
# emergency? if RTL fails, fall back to LAND
if not confirmed(action):
    set_mode_confirmed(other)  # cross-action
```

- **Idempotent** — redundant re-issue is harmless.
- **Zero client trust** — only telemetry confirms.
- **Emergencies too** — abort path as robust as nominal.
- **Proven in SITL** — silent-mode-reject bug mitigated.

Debounced, severity-ordered failsafe arbitration

Many hazards, evaluated at 1 Hz, resolve to exactly one calm recovery action.

- **Monotone severity** — actions ordered NONE < RTL < LAND; a LAND demand is never downgraded.
- **Debounced GPS** — fires only after N consecutive bad samples (reset on recovery); a glitch never lands the drone.
- **Mid-recovery escalation** — a critical-battery LAND overrides an in-progress RTL.
- **Fire-once discipline** — each hazard emits one event per mission (re-fire only to escalate).
- **Above firmware** — layered on unmodified ArduPilot/PX4 failsafes.

8 claims · pinned by a 37-case unit suite · IPC G05D 1/00, B64D 45/00, B64U 2201/10

The hazard tree: condition → threshold → action

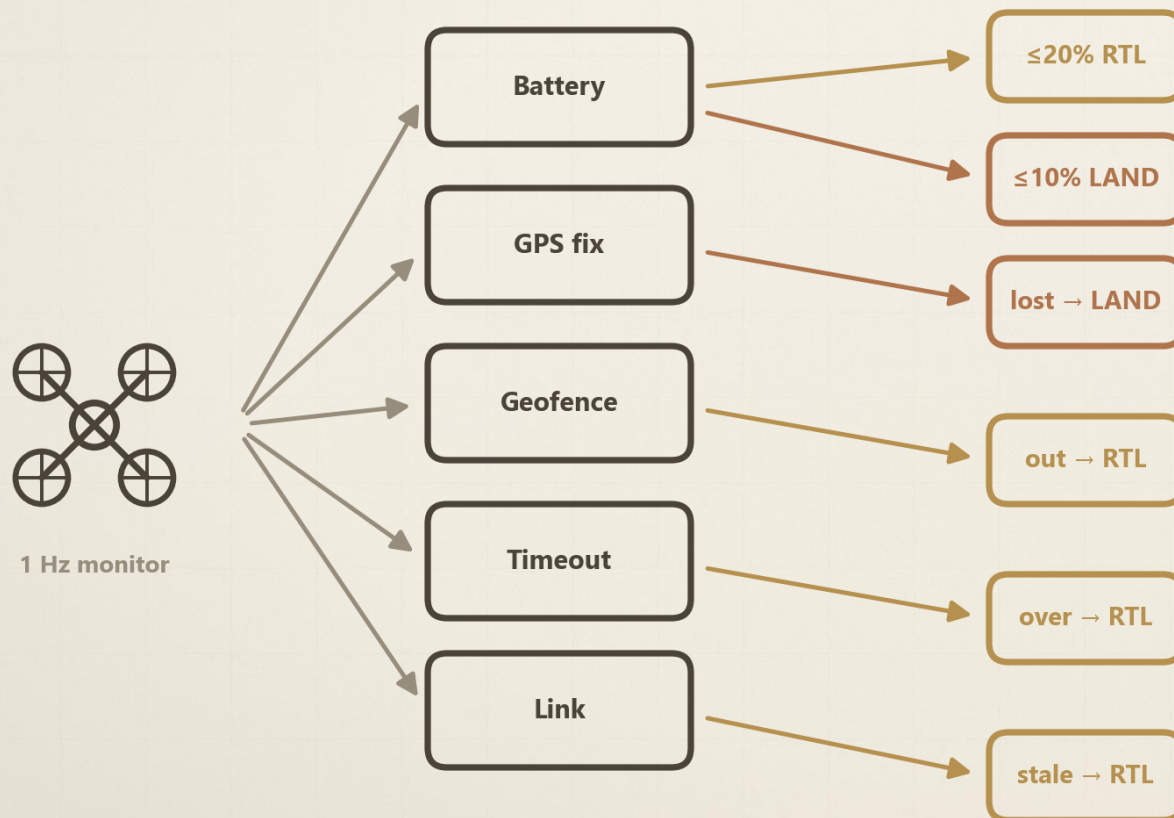


Figure 5 • Failsafe decision tree

- **RTL** — battery low, geofence, timeout, link loss.
- **LAND** — battery critical, GPS lost (can't navigate).
- **Debounce** — GPS needs N bad samples to fire.

Five gates from network trigger to landing



Figure 6 · The five-gate safety interlock

The guarantee:

even if four gates fail, no mission ever starts against an airborne drone.

Proven in software-in-the-loop

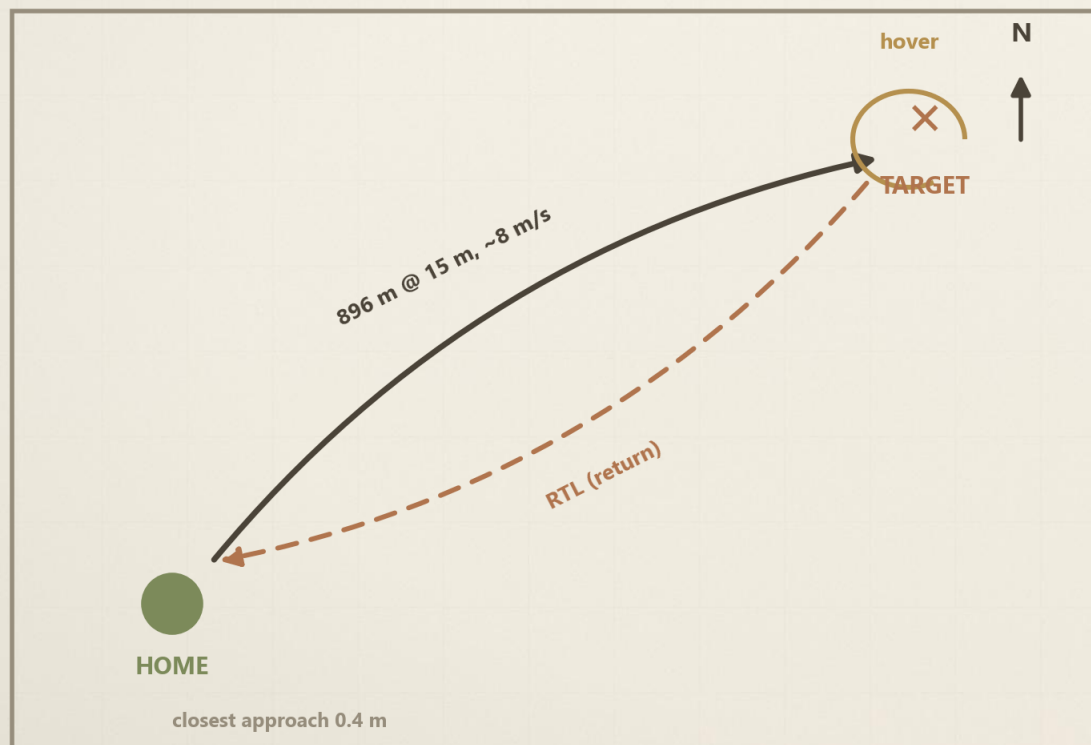


Figure 7 • Reconstructed flight path

8 / 8

*acceptance checks
pass*

0.4 m

*terminal accuracy (5
m tol.)*

37

*safety unit tests, ms-
fast*

328 s

*full autonomous
mission*

Technology stack

Flight core

- Python 3.11
- DroneKit + pymavlink
- MAVLink protocol
- ArduPilot SITL

Service

- FastAPI + Pydantic v2
- Uvicorn ASGI
- WebSocket telemetry
- SQLite store

Dashboard

- React 18 + Vite 5
- react-leaflet map
- Live path + log
- Follow-drone toggle

Quality / ops

- 37 pytest tests
- E2E SITL acceptance
- Docker compose
- API-key + geofence

Next steps & Claude Code integrations

Product hardening

- **Migrate off DroneKit** → MAVSDK / ArduPilot 4.x.
- **Failsafe-injection tests** — scripted SITL cuts.
- **Multi-leg routes** — full waypoint list up front.
- **RTK GPS** for ~0.1 m real-world accuracy.

Claude Code integrations

- **/fly slash command** — trigger → telemetry → cancel.
- **Save-hook** — auto-run unit tests on edits.
- **GitHub Actions CI** — run the safety suite per push.
- **MCP telemetry server** — query live state in chat.

Copyright vs. patent — what protects what

Copyright

Protects the expression — source code, docs, diagrams, the paper. Automatic on authorship; registration is cheap, fast, and yours to file now.

Patent

Protects the inventions — the verified setter, landing interlock, failsafe arbiter. Two IPO Form-2 drafts exist; needs a novelty search + a registered agent.

Recommended sequence

- **Now** — register copyright on the codebase + paper (immediate, low-cost).
- **Then** — run IPO InPASS / WIPO novelty search against both patent drafts.
- **Then** — file provisional patents via a registered agent before public disclosure.

Autonomous · Verified · Safe

Drone Safety System

A provable safety layer for autonomous UAV dispatch.

github.com/SV-1411/drone

Built with a companion-computer architecture · DroneKit · MAVLink · ArduPilot